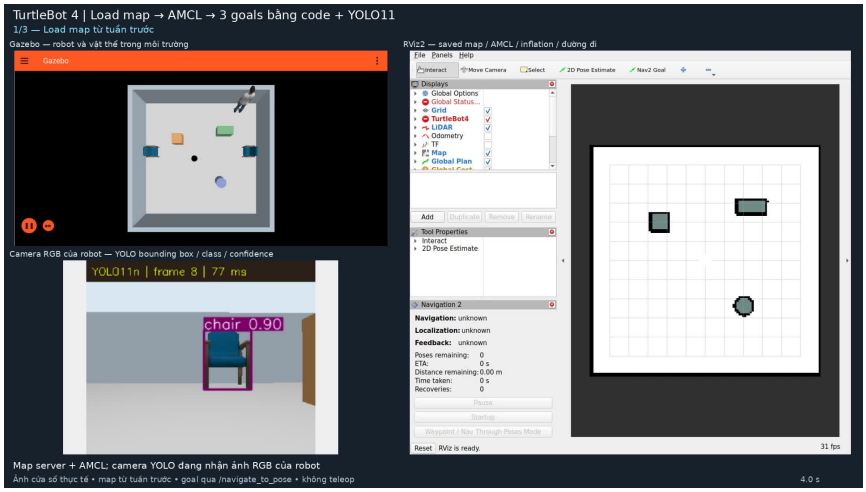
TRƯỜNG / KHOA .....

Môn học: Robot tự hành / Lập trình ROS 2

BÁO CÁO BÀI THỰC HÀNH

Định vị (AMCL), điều hướng tự động bằng Nav2 và nhận diện vật thể bằng YOLO trên TurtleBot 4 (Gazebo)



Sinh viên thực hiện: Trần Thắng
Email: thangtran2792005@gmail.com

Nền tảng: Ubuntu 22.04, ROS 2 Humble, Ignition Gazebo
Ngày hoàn thành: 14/10/2026

Video demo: Google Drive
GitHub: github.com/TranThang27/UR3_drawing (nhánh `turtlebot`)

Contents

1 Giới thiệu và mục tiêu **2**

2 Cấu trúc chương trình và luồng hoạt động **2**

2.1 Cấu trúc thư mục mã nguồn 2

2.2 Luồng hoạt động tổng thể 2

3 Các node, topic và action chính **3**

4 Phần 1 – Khởi chạy simulation, nạp bản đồ và định vị **3**

4.1 Môi trường Gazebo 3

4.2 Bản đồ được nạp lại 4

4.3 Khởi chạy AMCL và đặt initial pose 5

5 Phần 2 – Khảo sát và lấy tọa độ mục tiêu **5**

6 Phần 3 – Điều hướng tự động bằng code với Nav2 **6**

6.1 Node điều hướng 6

6.2 Nav2 lập kế hoạch và điều khiển như thế nào 7

6.3 Kết quả điều hướng 8

7 Phần 4 – Nhận diện vật thể bằng YOLO **8**

7.1 Node YOLO 8

7.2 Kết quả nhận diện 9

8 Hướng dẫn cài đặt và chạy **9**

9 Nhận xét và kết luận **10**

1 Giới thiệu và mục tiêu

Bài thực hành tuần này tái sử dụng bản đồ `map.yaml`/`map.pgm` đã xây dựng bằng SLAM Toolbox ở tuần trước để thực hiện ba việc trên robot TurtleBot 4 trong môi trường mô phỏng Gazebo:

1. **Localization**: nạp lại bản đồ, chạy AMCL và đặt initial pose sao cho dữ liệu LiDAR `/scan` trùng khớp với tường trên bản đồ (Phần 1);
2. **Navigation tự động bằng code**: một ROS 2 node Python gửi lần lượt ít nhất 3 goal tới action `/navigate_to_pose` của Nav2, không dùng teleop hay chuột đặt goal (Phần 2–3);
3. **Object detection**: node thứ hai subscribe ảnh RGB của camera OAK-D, chuyển sang OpenCV bằng `cv_bridge`, chạy YOLO11 và hiển thị/publish kết quả song song khi robot di chuyển (Phần 4).

Toàn bộ mã nguồn, bản đồ, cấu hình và hướng dẫn chạy được đưa lên GitHub (nhánh `turtlebot`); video demo được upload lên Google Drive. Các kết quả (số liệu, ảnh) trong báo cáo được trích từ một lần chạy hoàn chỉnh của chương trình và lưu trong thư mục `reports/nav_yolo_demo_20261014`.

Tóm tắt kết quả.

- Localization đạt: sai lệch vị trí AMCL so với Gazebo ngay sau khi đặt initial pose là 0,019 m và 0,93° (ngưỡng 0,2 m và 15°).
- Robot đi qua **3/3 goal** liên tiếp, cả ba trạng thái `SUCCEEDED`, **0** lần va chạm (bumper), **0** lệnh vận tốc thủ công.
- YOLO11n xử lý 378 khung hình, phát hiện đúng ghế (độ tin cậy cao nhất 0,93) và người (0,93) trong khi robot di chuyển.

2 Cấu trúc chương trình và luồng hoạt động

2.1 Cấu trúc thư mục mã nguồn

Mã nguồn nằm trong thư mục `turtlebot/` của repository. Chương trình không đóng gói thành `ament` package mà dùng `run.sh` làm điểm vào duy nhất: script source ROS 2 Humble, đặt `ROS_DOMAIN_ID=42`, dùng RMW CycloneDDS rồi gọi launch file hoặc script Python tương ứng.

```
turtlebot/
|-- run.sh                    # diem vao: ./run.sh {sim|localize|nav2|goals|yolo|yolo-demo|...}
|-- environment.sh            # ROS_DOMAIN_ID, CycloneDDS, overlay cho runtime
|-- launch/
|   |-- simulation.launch.py  # Gazebo (world simple_square) + TurtleBot 4 + RViz2
|   `-- navigation.launch.py  # map_server + AMCL (+ Nav2 neu navigation:=true)
|-- maps/map.yaml, map.pgm    # ban do tuan truoc (121x121 px, 0.05 m/px)
|-- config/goals.yaml         # 3 toa do muc tieu (x, y, yaw) trong frame map
|-- config/*.rviz             # cau hinh RViz2
|-- worlds/simple_square.sdf  # moi truong Gazebo phong 6x6 m
`-- scripts/
    |-- check_navigation.py   # NODE DIEU HUONG: gui Goal1->2->3 qua /navigate_to_pose
    |-- yolo_camera.py        # NODE YOLO: /oakd/rgb/preview/image_raw -> YOLO11
    |-- check_localization.py # dat initial pose + kiem chung AMCL vs Gazebo
    |-- add_demo_objects.py   # them ghe, nguoi vao Gazebo
    |-- run_yolo_demo.py      # chay toan bo quy trinh + quay video
    `-- setup_yolo.py         # cai torch (CPU), ultralytics, trong so yolo11n.pt
```

2.2 Luồng hoạt động tổng thể

Hình 1 mô tả luồng dữ liệu. Gazebo cung cấp `/scan`, `/odom` và ảnh camera qua `ros_gz_bridge`. AMCL kết hợp bản đồ do `map_server` phát với `/scan` và odometry để xuất phép biến đổi `map`→`odom`. Nav2 dùng TF này để lập kế hoạch toàn cục/cục bộ và xuất `/cmd_vel`. Node điều

hướng chỉ làm *action client*, còn node YOLO độc lập hoàn toàn với Nav2 và chỉ đọc camera nên hai node chạy song song.

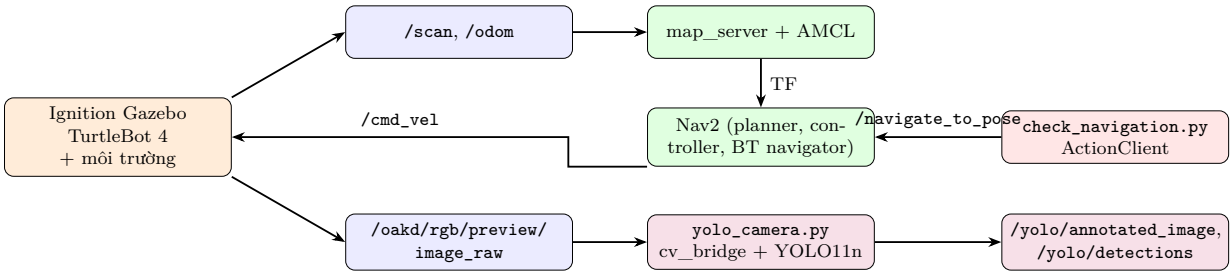


Figure 1: Sơ đồ khối: AMCL/Nav2 điều khiển robot; node YOLO chạy độc lập, song song.

Thứ tự thực thi trong lần chạy tự động `./run.sh yolo-demo`:

1. Khởi động Gazebo (world `simple_square`) và TurtleBot 4; thêm 2 ghế và 1 người đứng (model Gazebo Fuel) vào môi trường; tháo dock.
2. `./run.sh localize maps/map.yaml`: khởi động `map_server` và AMCL.
3. `check_localization.py -initial-pose -0.35 0 3.14159`: publish `/initialpose`, chờ AMCL xác nhận, so sánh TF `map`→`base_link` với pose thật của Gazebo.
4. `./run.sh nav2`: khởi động Nav2 với cấu hình chuẩn của TurtleBot 4.
5. `yolo_camera.py` chạy nền; `check_navigation.py -goals-file config/goals.yaml` gửi Goal 1→2→3.
6. Ghi lại `demo_results.json`, ảnh chụp và video.

3 Các node, topic và action chính

Thành phần	Package / script	Vai trò
`map_server`	`nav2_map_server`	Nạp `map.yaml`/`map.pgm`, phát `/map`
`amcl`	`nav2_amcl`	Định vị bằng particle filter, phát TF `map`→`odom` và `/amcl_pose`
`planner_server`	`nav2_planner`	Lập đường đi toàn cục (`/plan`) trên global costmap
`controller_server`	`nav2_controller`	Bám đường đi, tránh vật cản trên local costmap, xuất `/cmd_vel`
`bt_navigator`	`nav2_bt_navigator`	Behavior Tree; cung cấp action `/navigate_to_pose`
`tb4_navigation_check`	`check_navigation.py`	**Node điều hướng**: action client gửi 3 goal
`turtlebot4_yolo_camera`	`yolo_camera.py`	**Node YOLO**: nhận ảnh, suy luận, vẽ khung
`ros_gz_bridge`	`ros_gz_bridge`	Cầu nối Gazebo↔ROS 2 (scan, odom, ảnh, clock)

Table 1: Các node chính.

4 Phần 1 – Khởi chạy simulation, nạp bản đồ và định vị

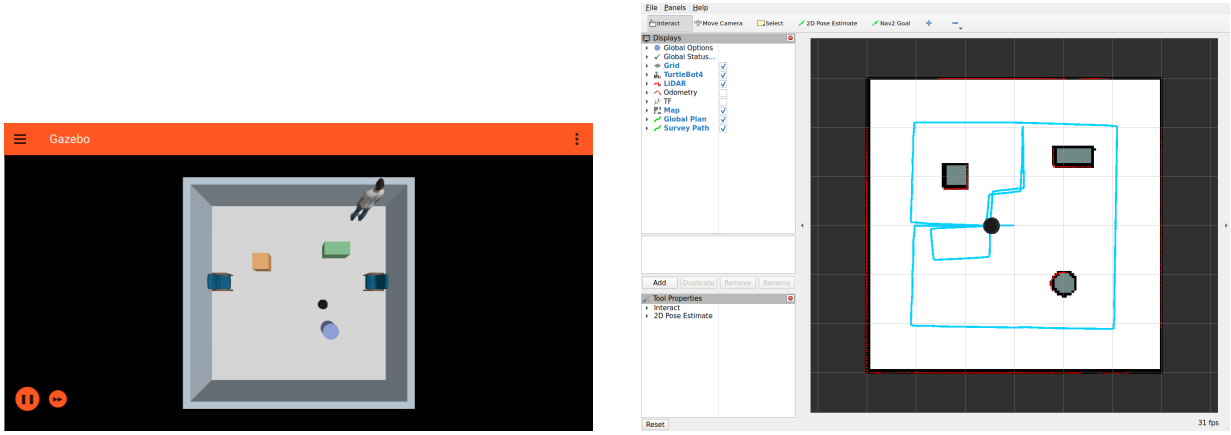
4.1 Môi trường Gazebo

Môi trường là phòng vuông 6×6 m (`worlds/simple_square.sdf`) đúng với world tuần trước dùng để quét bản đồ, gồm 2 hộp và 1 trụ làm vật cản tĩnh. Để phục vụ nhận diện, bổ sung thêm

Topic / Action	Kiểu message	Ghi chú
`/scan`	`sensor_msgs/LaserScan`	LiDAR RPLidar, quan sát ≈ 17 Hz
`/odom`	`nav_msgs/Odometry`	Odometry bánh xe ≈ 17 Hz
`/map`	`nav_msgs/OccupancyGrid`	Bản đồ từ `map_server`
`/initialpose`	`geometry_msgs/`	
`PoseWithCovarianceStamped`	Đặt pose ban đầu cho AMCL	
`/amcl_pose`	`geometry_msgs/`	
`PoseWithCovarianceStamped`	Pose ước lượng trong frame `map`	
`/plan`	`nav_msgs/Path`	Đường đi toàn cục Nav2
`/cmd_vel`	`geometry_msgs/Twist`	Lệnh vận tốc do Nav2 sinh ra
`/hazard_detection`	`irobot_create_msgs/`	
`HazardDetectionVector`	Bumper, dùng để đếm va chạm	
`/oakd/rgb/preview/`		
`image_raw`	`sensor_msgs/Image`	Ảnh RGB camera OAK-D (đầu vào YOLO)
`/yolo/annotated_image`	`sensor_msgs/Image`	Ảnh có bounding box (xem trên RViz2/rqt)
`/yolo/detections`	`std_msgs/String` (JSON)	Danh sách class, confidence, bbox
`/navigate_to_pose`		
(action)	`nav2_msgs/action/`	
`NavigateToPose`	Gửi goal (x, y, yaw) trong frame `map`	

Table 2: Topic và action chính.

2 ghế (ở $x = \pm 2{,}4$ m, $y = 0{,}35$ m) và 1 người đứng (1,95; 2,5) bằng `add_demo_objects.py`. Các vật này *không* có trong bản đồ nên Nav2 phải phát hiện chúng qua LiDAR và local costmap.



(a) Môi trường Gazebo nhìn từ trên xuống (ghế, người, hộp, trụ).

(b) Bản đồ tuần trước ngay sau khi quét bằng SLAM Toolbox.

Figure 2: Môi trường mô phỏng và bản đồ đã xây dựng.

4.2 Bản đồ được nạp lại

Bản đồ dùng đúng tệp của tuần trước (`maps/map.yaml`, `maps/map.pgm`); mã băm SHA-256 của hai tệp trước và sau lần chạy trùng nhau, chứng tỏ bản đồ không bị sửa.

```
image: map.pgm        resolution: 0.05      origin: [-3.03, -3.03, 0.0]
mode: trinary         occupied_thresh: 0.65 free_thresh: 0.196   negate: 0
```

Bản đồ có kích thước 121 × 121 pixel, tức 6,05 × 6,05 m, diện tích trống 33,9 m^2.

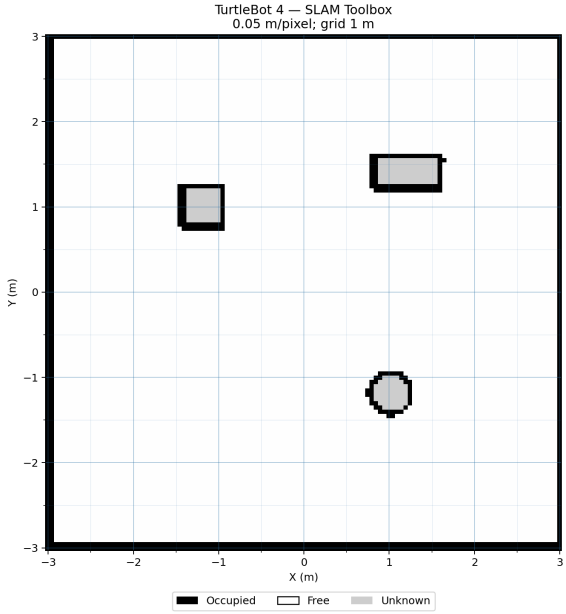


Figure 3: Bản đồ occupancy grid (`map.pgm`) với lưới 1 m: tường bao quanh, 2 vật hình chữ nhật và 1 vật hình tròn.

4.3 Khởi chạy AMCL và đặt initial pose

Lệnh `./run.sh localize maps/map.yaml` gọi `navigation.launch.py` với `navigation:=false`, tức chỉ khởi động `map_server` và `amcl` từ package `turtlebot4_navigation`. Robot xuất phát tại $(-0{,}35;\ 0;\ \pi)$ trong frame `map` (trục map trùng trục world vì SLAM bắt đầu từ gốc world). Initial pose được publish lên `/initialpose` với hiệp phương sai 0,25 m^2 ở x, y và 0,1 rad^2 ở yaw, đúng như thao tác *2D Pose Estimate* trên RViz2 nhưng thực hiện bằng code để lặp lại được:

```
seed = PoseWithCovarianceStamped()
seed.header.frame_id = 'map'
seed.pose.pose.position.x, seed.pose.pose.position.y, yaw = args.initial_pose
seed.pose.pose.orientation.z, seed.pose.pose.orientation.w = math.sin(yaw/2), math.cos(yaw/2)
seed.pose.covariance[0] = seed.pose.covariance[7] = .25
seed.pose.covariance[35] = .1
initial.publish(seed)   # lap lai moi 2 s cho den khi AMCL xac nhan bang /amcl_pose moi
```

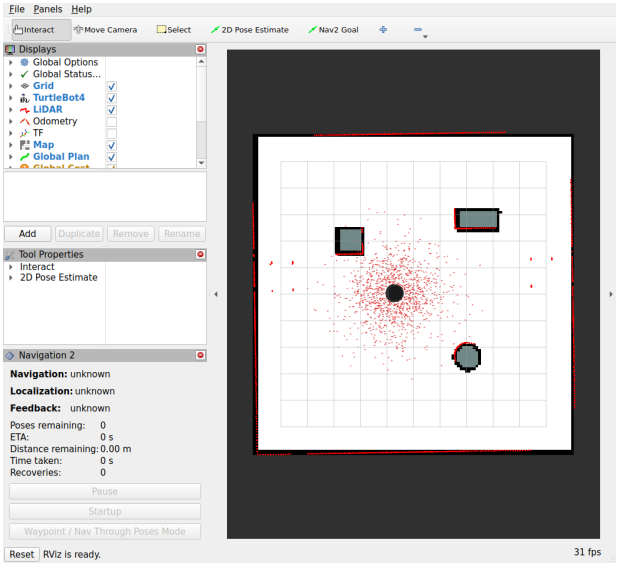
Ngay sau khi nhận initial pose, particle cloud (các chấm đỏ nhỏ quanh robot trên Hình 4a) hội tụ dần, các điểm LiDAR (đỏ) nằm khít lên các cạnh tường và hai vật trên bản đồ. Để kiểm chứng định lượng, `check_localization.py` so sánh TF `map`→`base_link` do AMCL xuất với pose thật lấy từ Gazebo (chỉ dùng để *đo*, không đưa vào AMCL):

	x (m)	y (m)	yaw (rad)
Gazebo (ground truth)	−0,3526	0,0000	3,1351
AMCL (map→base_link)	−0,3492	0,0185	−3,1319
Sai lệch	**0,019 m**		**0,016 rad** (0,93°)

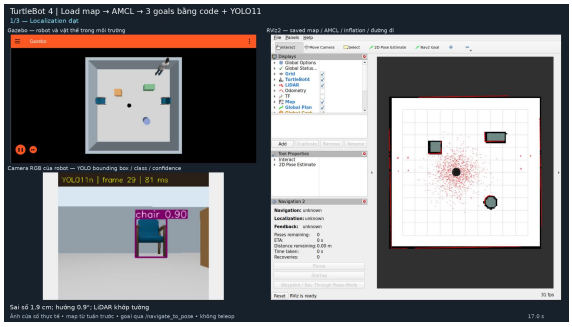
Table 3: Kiểm chứng localization sau khi đặt initial pose (đạt: $< 0{,}2$ m và $< 15°$; yaw $+\pi$ và $-\pi$ là cùng một hướng).

5 Phần 2 – Khảo sát và lấy tọa độ mục tiêu

Tọa độ được xác định trên bản đồ bằng công cụ *Publish Point* của RViz2 (đọc qua `ros2 topic echo /clicked_point`) và đối chiếu với `/amcl_pose`; yaw chọn sao cho robot quay mặt vào phía



(a) RViz2: map, LiDAR (đỏ) và particle cloud AMCL.



(b) Cửa sổ tổng hợp: Gazebo, camera YOLO và RViz2.

Figure 4: Kết quả định vị: bản đồ hiển thị đúng, `/scan` trùng tường trên bản đồ.

trong phòng. Cả ba điểm nằm ở các góc khác nhau, cách tường $\geq 1{,}2$ m và cách vật cản tĩnh > 1 m để nằm ngoài vùng inflation.

Goal	**Khu vực**	x (m)	y (m)	yaw (rad)
Goal 1	Đông–Nam	1,8	−1,8	1,40
Goal 2	Tây–Nam	−1,8	−1,8	1,80
Goal 3	Tây–Bắc	−1,8	2,0	0,12

Table 4: Ba goal trong hệ tọa độ `map`, lưu tại `config/goals.yaml`.

```
frame_id: map
goals:
  - name: Goal 1 - southeast    pose: [1.8, -1.8, 1.4]
  - name: Goal 2 - southwest    pose: [-1.8, -1.8, 1.8]
  - name: Goal 3 - northwest    pose: [-1.8, 2.0, 0.12]
```

6 Phần 3 – Điều hướng tự động bằng code với Nav2

6.1 Node điều hướng

Node `tb4_navigation_check` (`scripts/check_navigation.py`) được viết bằng `rclpy`, dùng trực tiếp `ActionClient(NavigateToPose, '/navigate_to_pose')` thay cho `nav2_simple_commander`. Điều này cho phép đọc trực tiếp trạng thái kết quả, TF cuối cùng, số bản tin `/plan` và `/hazard_detection` cho từng goal. Luồng xử lý:

1. Đọc `goals.yaml`, kiểm tra tối thiểu 3 goal trong frame `map`.
2. Chờ AMCL có `/amcl_pose` và TF `map`→`base_link`; chờ action server và `bt_navigator` ở trạng thái *active* (lifecycle id 3).
3. Với từng goal: tạo `NavigateToPose.Goal`, gửi, chờ kết quả (timeout 360 s), tra TF cuối để tính sai số vị trí/yaw.
4. Goal chỉ được coi là đạt khi status `SUCCEEDED`, sai số vị trí $< 0{,}35$ m, có kế hoạch `/plan`, và *không* có bản tin bumper chạm. Nếu không đạt thì dừng và báo lỗi.
5. Nghỉ (`-dwell`) rồi sang goal kế tiếp; cuối cùng ghi báo cáo JSON.

```
client = ActionClient(node, NavigateToPose, '/navigate_to_pose')
```

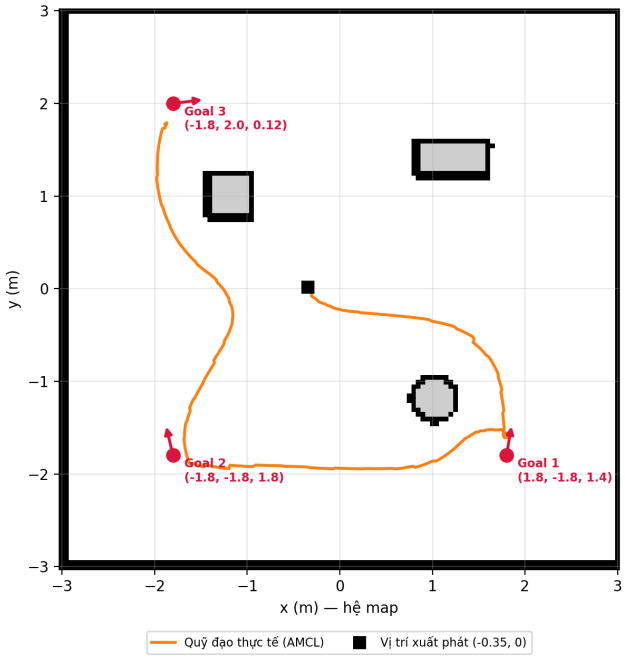


Figure 5: Bản đồ kèm ba điểm Goal (chấm đỏ, mũi tên là hướng yaw) và quỹ đạo thực tế của robot (cam) do AMCL ghi lại trong lần chạy; ô vuông đen là vị trí xuất phát.

```
for x, y, yaw in args.goal:
    goal = NavigateToPose.Goal()
    goal.pose.header.frame_id = 'map'
    goal.pose.header.stamp = node.get_clock().now().to_msg()
    goal.pose.pose.position.x, goal.pose.pose.position.y = x, y
    goal.pose.pose.orientation.z = math.sin(yaw / 2)      # yaw -> quaternion
    goal.pose.pose.orientation.w = math.cos(yaw / 2)
    sent = client.send_goal_async(goal)
    wait(sent.done, 15)
    handle = sent.result()                                # goal accepted?
    result = handle.get_result_async()
    wait(result.done, args.timeout)
    entry['status'] = result.result().status              # 4 = SUCCEEDED
```

Sau khi chạy node này không có bất kỳ thao tác nào khác: không teleop, không đặt goal bằng chuột trên RViz2 (`manual_velocity_commands_during_mission = 0`).

6.2 Nav2 lập kế hoạch và điều khiển như thế nào

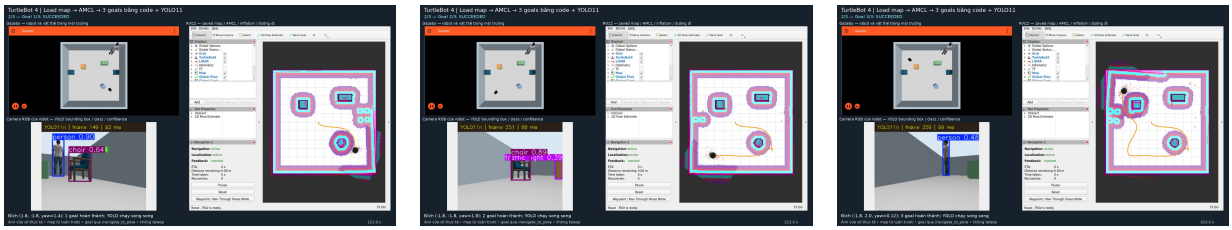
- **Global planner** (`NavfnPlanner`, thuật toán Dijkstra/A*) tìm đường ngắn nhất trên *global costmap* gồm static layer (từ `/map`), obstacle layer (LiDAR) và inflation layer; đường đi được publish trên `/plan` (đường cam trên RViz2, Hình 7).
- **Local controller** bám theo đường đi ở tần số cao trên *local costmap* cửa sổ trượt quanh robot, chọn vận tốc (v, ω) tránh vật cản mới xuất hiện (ghế, người) rồi publish `/cmd_vel`.
- **Inflation** (vùng màu hồng/tím quanh vật cản trên Hình 7) làm tăng chi phí gần vật cản nên đường đi giữ khoảng cách an toàn; hai ghế ở $x = \pm 2{,}4$ m xuất hiện như vật cản mới trên local costmap dù không có trong bản đồ.
- **Behavior Tree** trong `bt_navigator` điều phối: tính đường → bám đường → nếu kẹt thì chạy *recovery* (xoay, lùi, xóa costmap). Trong lần chạy này số lần recovery bằng 0 (`Recoveries: 0`).

Goal	Trạng thái	Sai số vị trí	Sai số yaw	/plan	Bumper	Thời gian
1	SUCCEEDED	0,184 m	0,248 rad	86	0	≈ 88 s
2	SUCCEEDED	0,230 m	0,256 rad	83	0	≈ 85 s
3	SUCCEEDED	0,248 m	0,249 rad	88	0	≈ 91 s

Table 5: Kết quả ba goal liên tiếp (thời gian lấy từ mốc video, mô phỏng thời gian thực; sai số nằm trong dung sai goal mặc định của Nav2 ≈ 0,25 m / 0,25 rad).

6.3 Kết quả điều hướng

Sau goal cuối, kiểm tra độc lập so với Gazebo cho sai lệch AMCL là 0,116 m và 0,071 rad, nghĩa là robot không chỉ "tin" là đã đến nơi mà thực sự ở gần vị trí đó.



(a) Goal 1 đạt **(b)** Goal 2 đạt **(c)** Goal 3 đạt

Figure 6: Ảnh chụp từ video tại thời điểm hoàn thành từng goal (Gazebo, camera YOLO, RViz2 với đường đi cam, costmap, `Feedback: reached`).

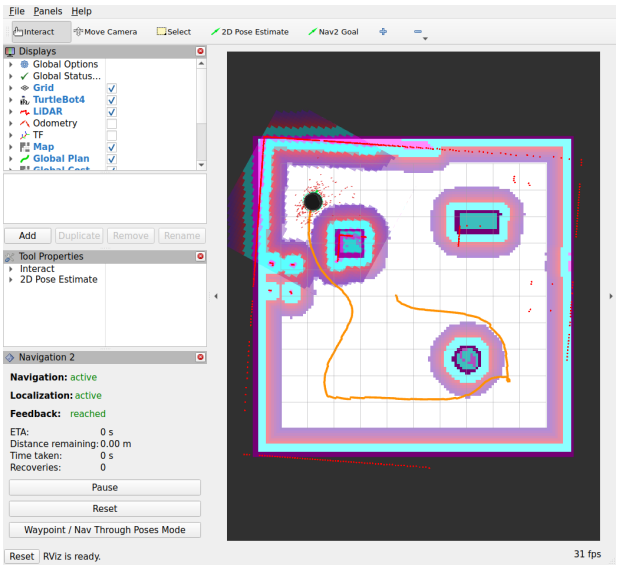


Figure 7: RViz2 sau khi đi hết ba goal: bản đồ, global costmap có vùng inflation, đường đi toàn cục (cam), điểm LiDAR (đỏ). `Navigation: active`, `Localization: active`, `Recoveries: 0`.

7 Phần 4 – Nhận diện vật thể bằng YOLO

7.1 Node YOLO

Node `turtlebot4_yolo_camera` (`scripts/yolo_camera.py`) dùng mô hình **YOLO11n** (Ultralytics, trọng số COCO `yolo11n.pt`) chạy trên CPU với PyTorch 2.5.1. Các bước xử lý mỗi khung hình:

1. Subscribe `/oakd/rgb/preview/image_raw` (QoS `sensor_data`); chỉ giữ khung mới nhất để không bị trễ tích lũy.

2. Timer 5 Hz: chuyển `sensor_msgs/Image`→OpenCV (`bgr8`) bằng `CvBridge.imgmsg_to_cv2`.
3. `model.predict(image, imgsz=640, conf=0.3)`.
4. Vẽ bounding box, tên class, confidence bằng `result.plot()`, thêm thanh tiêu đề (số khung, thời gian suy luận).
5. Publish ảnh lên `/yolo/annotated_image` (xem được trong RViz2), publish JSON lên `/yolo/detections`, hiển thị cửa sổ OpenCV (`-display`), in log ra terminal và lưu `detections.jsonl`.

```
self.subscription = self.create_subscription(
    Image, args.topic, self.receive, qos_profile_sensor_data)
...
image = self.bridge.imgmsg_to_cv2(message, desired_encoding='bgr8')
result = self.model.predict(image, imgsz=self.args.imgsz,
                            conf=self.args.conf, device='cpu', verbose=False)[0]
for box in result.boxes:
    name = result.names[int(box.cls.item())]
    objects.append({'class': name, 'confidence': float(box.conf.item()),
                    'xyxy': [float(v) for v in box.xyxy[0].tolist()]})
annotated = cv2.copyMakeBorder(result.plot(), 30, 0, 0, 0, cv2.BORDER_CONSTANT, value=(25,32,42))
self.annotated.publish(self.bridge.cv2_to_imgmsg(annotated, encoding='bgr8'))
self.detections.publish(String(data=json.dumps(payload)))
```

Thông tin phát hiện được in ra terminal mỗi 2 s, ví dụ:

```
[turtlebot4_yolo_camera]: YOLO ready: yolo11n.pt; input /oakd/rgb/preview/image_raw
[turtlebot4_yolo_camera]: frame=1: chair 0.90
[turtlebot4_yolo_camera]: frame=161: person 0.93
```

7.2 Kết quả nhận diện

Trong lần chạy, node nhận 2 285 khung và xử lý 378 khung (5 Hz), trong đó 199 khung có ít nhất một vật thể; 325 khung xử lý khi robot đang điều hướng, 146 khung trong số đó có phát hiện. Thời gian suy luận ≈ 80–110 ms/khung trên CPU. Hai class chính xác là *chair* và *person*. Ba

Class	Số lần phát hiện	Conf. cao nhất	Nhận xét
chair	134	0,926	đúng (2 ghế trong môi trường)
person	77	0,929	đúng (người đứng)
bench	19	0,700	ghế nhìn nghiêng bị gán nhãn trùng
traffic light	10	0,575	nhận nhầm cạnh hộp/tường
umbrella	4	0,346	nhận nhầm góc tường

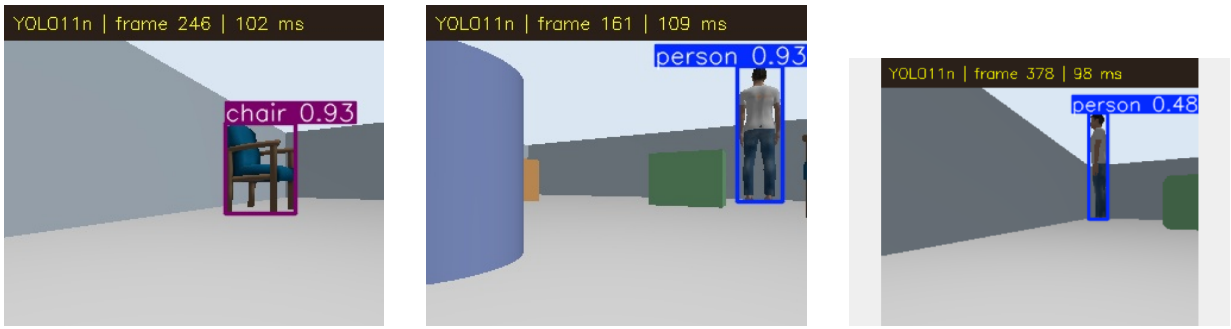
Table 6: Thống kê phát hiện của YOLO11n (ngưỡng conf = 0,3).

class còn lại có độ tin cậy thấp hơn và thực chất là nhận nhầm (Hình 9); nguyên nhân là mô hình COCO được huấn luyện trên ảnh thật, trong khi mô phỏng chỉ có các khối hình học đơn giản. Có thể giảm nhầm lẫn bằng cách tăng ngưỡng `-conf` lên 0,5 hoặc dùng YOLO-World với tập class giới hạn.

8 Hướng dẫn cài đặt và chạy

```
git clone -b turtlebot https://github.com/TranThang27/UR3_drawing.git
cd UR3_drawing/turtlebot
sudo apt install ros-humble-turtlebot4-simulator ros-humble-turtlebot4-navigation \
  ros-humble-slam-toolbox ros-humble-nav2-bringup ros-humble-cv-bridge \
  ros-humble-rmw-cyclonedds-cpp ros-humble-rviz2 ffmpeg wmctrl
./run.sh setup && ./run.sh doctor     # tai runtime, kiem tra package
./run.sh setup-yolo                   # torch CPU + ultralytics + yolo11n.pt

# Cach 1: tu dong toan bo (load map -> AMCL -> 3 goal bang code -> YOLO -> quay video)
./run.sh yolo-demo --output reports/nav_yolo_demo_new
```



(a) chair 0,93 **(b)** person 0,93 **(c)** Cửa sổ OpenCV khi chạy

Figure 8: Kết quả nhận diện đúng: bounding box, tên class, độ tin cậy.

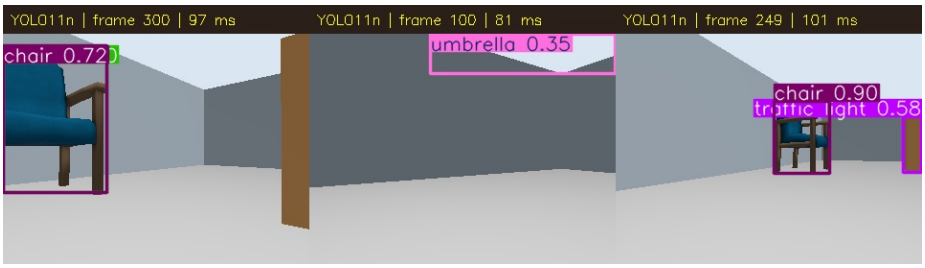


Figure 9: Các phát hiện kém tin cậy (trái→phải): bench 0,72 (cùng chiếc ghế), umbrella 0,35 (góc tường), traffic light 0,58 (cạnh hộp).

```
# Cach 2: tung buoc
./run.sh sim rviz_config:="$PWD/config/localization.rviz"     # terminal 1
./run.sh undock; ./run.sh remove-dock                         # terminal 2
./run.sh localize maps/map.yaml                               # terminal 3 (AMCL)
./run.sh nav2                                                 # terminal 4 (Nav2)
./run.sh yolo --display                                       # terminal 5 (YOLO)
./run.sh goals --goals-file config/goals.yaml --dwell 6       # terminal 6 (3 goal)
```

9 Nhận xét và kết luận

Kết quả đạt được. Đã nạp lại bản đồ cũ và định vị thành công bằng AMCL (sai lệch 0,019 m); robot tự đi qua 3 goal liên tiếp bằng node Python gọi action `/navigate_to_pose`, không va chạm, không can thiệp thủ công; node YOLO11n chạy song song, nhận đúng ghế và người với độ tin cậy trên 0,9 và publish kết quả lên `/yolo/annotated_image`.

Hạn chế.

- Sai số yaw tại goal ≈ 0,25 rad do dung sai mặc định của Nav2; có thể giảm bằng cách siết `yaw_goal_tolerance` trong `nav2.yaml`.
- YOLO chạy trên CPU (∼ 10 FPS lý thuyết, cấu hình 5 Hz); có nhận nhầm class với các khối hình học đơn giản.
- Initial pose cần biết trước vị trí xuất phát; nếu không biết, có thể dùng `/reinitialize_global_localizat` của AMCL.
- Mỗi goal mất khoảng 85–90 s vì robot chạy chậm và Gazebo chạy trên CPU/GPU tích hợp.

Hướng phát triển. Gắn vị trí vật thể phát hiện vào bản đồ bằng ảnh depth của OAK-D; dùng `nav2_waypoint_follower` để đi qua nhiều điểm không dừng; thay YOLO11n bằng YOLO-World để nhận diện theo từ khóa.

Liên kết

- Video demo (Google Drive): https://drive.google.com/drive/folders/1LF-GkSyY50TnkUzGGbq_RCvAMGTMZOS7?hl=vi
- GitHub (nhánh `turtlebot`, thư mục `turtlebot/`): https://github.com/TranThang27/UR3_drawing/tree/turtlebot/turtlebot